

Rezolvare Completă și Detaliată

Concursul Național pentru Ocuparea Posturilor Didactice

Anul 2018 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Titularizare Varianta 3 (11 iulie 2018)	2
SUBIECTUL I (30 de puncte)	2
1. Algoritmica recursivității	2
2. Protecția și securitatea sistemelor de calcul	2
SUBIECTUL al II-lea (30 de puncte)	2
1. Programare: Căutare în listă simplu înlănțuită ordonată	2
2. Algoritm eficient: Cel mai lung subșir comun (LCS $O(M \cdot N)$ timp/spațiu)	5

I. Rezolvare Titularizare Varianta 3 (11 iulie 2018)

[Subiect PDF](file:

SUBIECTUL I (30 de puncte)

1. Algoritmica recursivității

• Noțiuni:

- **Recursivitate directă:** Subprogramul se autoapelează direct din corpul său.
- **Recursivitate indirectă:** Subprogramul A apelează subprogramul B, iar B la rândul său apelează pe A.

• **Mecanism:** Folosește stiva sistemului (Stack) pentru a reține adresa de revenire, parametrii transmiși și variabilele locale ale fiecărui nivel de autoapel.

• **Avantaje:** Cod mai scurt, mai clar, ușor de înțeles pentru algoritmi de tip Divide et Impera sau Backtracking.

• **Dezavantaje:** Consum suplimentar de memorie (risc de Stack Overflow), timp de execuție mai lung din cauza overhead-ului de apel.

• Problemă (Determinare c.m.m.d.c.):

```
#include <iostream>
using namespace std;
int cmmdc(int a, int b) {
    if (b == 0) return a;
    return cmmdc(b, a % b);
}
```

—

2. Protecția și securitatea sistemelor de calcul

• **Noțiuni:** Sistem de calcul (hardware + software), SO (interfața utilizator-hardware), Rețea (sisteme interconectate).

• Elemente de securitate:

1. **Firewall:** Filtrează traficul de date la nivel de porturi/adrese IP.
2. **Antivirus/Antimalware:** Scanează fișierele pe baza semnăturilor de viruși și comportamentului euristic.
3. **Criptare/Control acces:** Protejează integritatea și confidențialitatea datelor (ex. algoritmi de criptare simetrică/asimetrică, parole).

—

SUBIECTUL al II-lea (30 de puncte)

1. Programare: Căutare în listă simplu înlănțuită ordonată

Soluție C++:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Nod {
    int info;
    Nod* urm;
};
```

```

Nod* caut(Nod* p, int x) {
    Nod* rez = nullptr;
    Nod* curr = p;
    while (curr != nullptr && curr->info <= x) {
        rez = curr;
        curr = curr->urm;
    }
    return rez;
}

void insertSorted(Nod* &p, int x) {
    Nod* nou = new Nod{x, nullptr};
    if (p == nullptr || x < p->info) {
        nou->urm = p;
        p = nou;
        return;
    }
    Nod* curr = p;
    while (curr->urm != nullptr && curr->urm->info < x) {
        curr = curr->urm;
    }
    nou->urm = curr->urm;
    curr->urm = nou;
}

int main() {
    int n;
    if (!(cin >> n)) return 0;

    Nod* p = nullptr;
    for (int i = 0; i < n; ++i) {
        int val;
        cin >> val;
        insertSorted(p, val);
    }

    // Afişare listă
    for (Nod* curr = p; curr != nullptr; curr = curr->urm) {
        cout << curr->info << (curr->urm == nullptr ? "" : " ");
    }
    cout << endl;

    return 0;
}

```

Soluție Pascal:

```

program CautareLista;
type
    PNod = ^TNod;
    TNod = record
        info: integer;
        urm: PNod;
    end;
var
    p, curr: PNod;

```

```

n, i, val: integer;

function caut(p_list: PNode; x: integer): PNode;
var
    curr_node, rez: PNode;
begin
    rez := nil;
    curr_node := p_list;
    while (curr_node <> nil) and (curr_node^.info <= x) do
    begin
        rez := curr_node;
        curr_node := curr_node^.urm;
    end;
    caut := rez;
end;

procedure insertSorted(var p_list: PNode; x: integer);
var
    nou, curr_node: PNode;
begin
    new(nou);
    nou^.info := x;
    nou^.urm := nil;
    if (p_list = nil) or (x < p_list^.info) then
    begin
        nou^.urm := p_list;
        p_list := nou;
        exit;
    end;
    curr_node := p_list;
    while (curr_node^.urm <> nil) and (curr_node^.urm^.info < x) do
        curr_node := curr_node^.urm;
    nou^.urm := curr_node^.urm;
    curr_node^.urm := nou;
end;

begin
    readln(n);
    p := nil;
    for i := 1 to n do
    begin
        read(val);
        insertSorted(p, val);
    end;

    curr := p;
    while curr <> nil do
    begin
        write(curr^.info);
        if curr^.urm <> nil then write(' ');
        curr := curr^.urm;
    end;
    writeln;
end.

```

—

2. Algoritm eficient: Cel mai lung subșir comun (LCS $O(M \cdot N)$ timp/spațiu)

Soluție C++:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    ifstream fin("titu.in");
    int m, n;
    if (!(fin >> m >> n)) return 0;
    vector<int> A(m), B(n);
    for (int i = 0; i < m; ++i) fin >> A[i];
    for (int i = 0; i < n; ++i) fin >> B[i];
    fin.close();

    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            if (A[i-1] == B[j-1]) {
                dp[i][j] = dp[i-1][j-1] + 1;
            } else {
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
            }
        }
    }

    cout << dp[m][n] << endl;
    return 0;
}
```

Soluție Pascal:

```
program SubsirComunMaxim;
var
    fin: text;
    m, n, i, j: integer;
    A, B: array[1..100] of integer;
    dp: array[0..100, 0..100] of integer;
begin
    assign(fin, 'titu.in'); reset(fin);
    read(fin, m, n);
    for i := 1 to m do read(fin, A[i]);
    for i := 1 to n do read(fin, B[i]);
    close(fin);

    for i := 0 to m do
        for j := 0 to n do
            dp[i, j] := 0;

    for i := 1 to m do
        begin
            for j := 1 to n do
                begin
                    if A[i] = B[j] then
```

```
    dp[i, j] := dp[i - 1, j - 1] + 1
else
begin
    if dp[i - 1, j] > dp[i, j - 1] then
        dp[i, j] := dp[i - 1, j]
    else
        dp[i, j] := dp[i, j - 1];
    end;
end;
end;
writeln(dp[m, n]);
end.
```